Sharing R Markdown Code Reproducibly with Docker

A practical walkthrough of Dockerizing an R analysis for reliable collaboration

Ronald 'Ryy' G. Thomas

2025-06-25

Table of contents

1 Introduction **2**
- 1.1 Motivations . . . 3
- 1.2 Objectives . . . 3

2 Prerequisites and Setup **4**

3 What is Docker for R Users? **4**

4 Getting Started: The Naive Approach **5**
- 4.1 The Analysis . . . 5
- 4.2 Sending the File to Joe . . . 7
- 4.3 Error 1: R Not Found . . . 7
- 4.4 Error 2: Function render Not Found . . . 7
- 4.5 Error 3: Pandoc Version Too Old . . . 7
- 4.6 Error 4: Package pacman Not Found . . . 8
- 4.7 Error 5: Missing preamble.tex . . . 8
- 4.8 Error 6: pdflatex Not Found . . . 8
- 4.9 Error 7: Missing Logo File . . . 8
- 4.10 Error 8: Missing Bibliography File . . . 8
- 4.11 Error 9: Package Loading Failures . . . 8
- 4.12 Finally: Success . . . 9

5 The Docker Approach **11**
- 5.1 The Dockerfile . . . 11
- 5.2 Building the Image . . . 12
- 5.3 Sharing the Image . . . 12
- 5.4 Running the Container . . . 12
- 5.5 The LaTeX Preamble . . . 13
- 5.6 Things to Watch Out For . . . 14
- 5.7 Lessons Learnt . . . 14
  - 5.7.1 Conceptual Understanding . . . 14

5.7.2 Technical Skills 15
5.7.3 Gotchas and Pitfalls 15
5.8 Limitations 15
5.9 Opportunities for Improvement 16

6 Wrapping Up **16**

7 See Also **16**

8 Reproducibility **17**

9 Let's Connect **17**

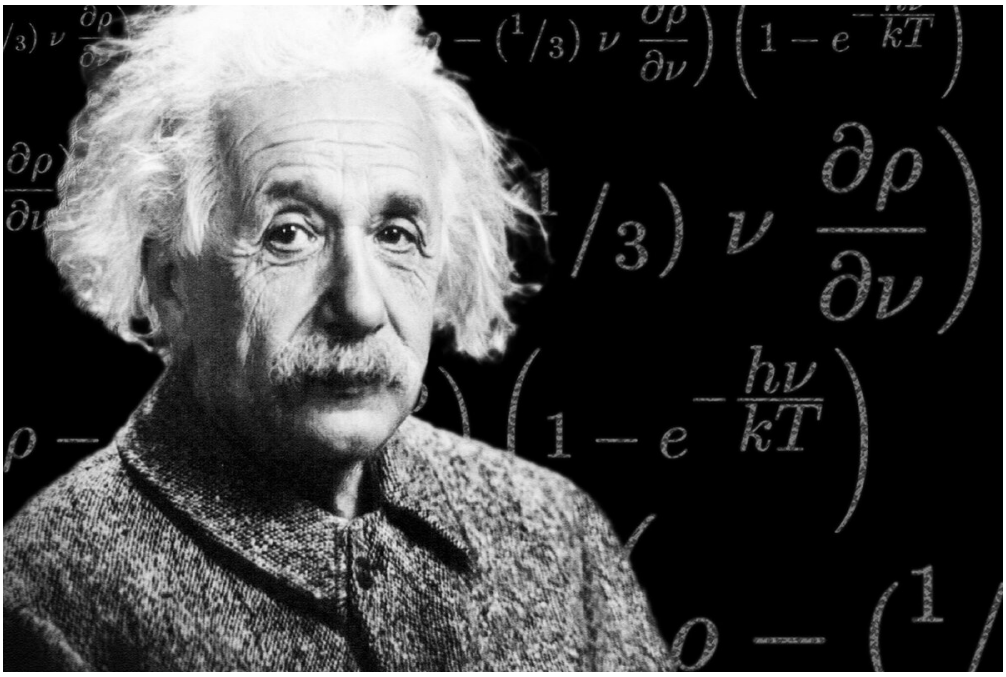

Figure 1: A shipping container yard representing the concept of containerised, portable software environments.

Just as shipping containers standardised global freight, Docker containers standardise software delivery.

1 Introduction

I did not really know how fragile sharing R code could be until my colleague – let us call him Joe – spent an entire afternoon debugging missing packages, incompatible R versions, and absent LaTeX files, all because I sent him a single `.Rmd` file by email. Every error he encountered was something I could have prevented by packaging my analysis in a Docker container.

The experience was instructive because the code ran perfectly on my machine. I had the right R version, the right packages at the right versions, the right LaTeX distribution with the right

preamble file, and even a custom logo image that the report referenced. None of these were available on Joe's Linux Mint workstation. Every missing piece produced a different, cryptic error message.

This post walks through both approaches: the naive method of emailing an Rmd file (and watching it fail), and the Docker method that eliminates every one of those failures. The narrative is based on a real collaboration, and the sequence of errors Joe encountered is presented in the order they actually occurred.

More formally, this post documents the Container layer of the Workflow Construct described in post 52, with R Markdown as the worked example. The Container layer is replaceable across runtimes (Docker on the laptop, Apptainer on HPC, Podman as a daemonless alternative); its core machinery (Dockerfile authoring, base image selection, dependency installation, port binding) is the same regardless of what the container ultimately serves. This post is the canonical container-layer post for the construct; post 33 is a Shiny-specific case study that defers to this post for layer fundamentals.

1.1 Motivations

- I had sent an R Markdown analysis file to a colleague and watched him spend hours debugging environment issues that had nothing to do with the analysis itself.
- Every error my colleague encountered – missing R, missing packages, missing LaTeX, missing auxiliary files – was entirely preventable with proper packaging.
- I wanted a reproducible method for sharing R analyses that would work regardless of the recipient's operating system, R version, or installed packages.
- I was aware of Docker conceptually but had never used it specifically for sharing R Markdown analyses, and I wanted a practical, end-to-end example.
- The experience convinced me that "works on my machine" is not a valid standard for collaborative data science.

1.2 Objectives

1. Demonstrate the failure modes of the naive approach to sharing R Markdown code by walking through the sequence of errors a collaborator encounters.
2. Write a complete Dockerfile that packages an R Markdown analysis with all its dependencies, including R packages, LaTeX, auxiliary files, and data.
3. Build, push, and pull a Docker image so that a collaborator can render the analysis with a single command.
4. Document the complete workflow from authoring to collaboration, including volume mounting for output files.

I am documenting my learning process here. If you spot errors or have better approaches, please let me know.

The analysis begins on one machine, but it needs to run on many.

2 Prerequisites and Setup

Before proceeding, you will need:

- **Docker Desktop** installed on your machine (macOS, Windows, or Linux)
- **R 4.0+** and the `rmarkdown` package installed locally (for authoring)
- **A Docker Hub account** (free) for pushing and sharing images
- Basic familiarity with the terminal and R Markdown document structure

If you have never used Docker before, the only concept you need is this: a Docker image is a snapshot of a complete computing environment (operating system, software, files), and a Docker container is a running instance of that image. When you share an image, the recipient gets an exact copy of your environment.

3 What is Docker for R Users?

Docker is a tool that packages an application and its entire computing environment – operating system libraries, programming language, installed packages, configuration files, data – into a single portable unit called an image. When someone runs that image, they get a container that behaves identically to the environment in which you developed your analysis.

Think of it like shipping a complete laboratory instead of just a lab notebook. The naive approach to sharing code is like sending someone your lab notes and hoping they have the same equipment, reagents, and room temperature. Docker is like shipping the entire lab bench, pre-loaded with everything needed to replicate your experiment.

For R users specifically, Docker solves the problem of environment divergence. Your R version, your installed packages, your LaTeX distribution, your auxiliary files, and your data are all captured in the image. The recipient does not need to install anything except Docker itself.

4 Getting Started: The Naive Approach

To understand why Docker matters, it is instructive to see what happens when you share R Markdown code without it. The following narrative is based on a real collaboration.

4.1 The Analysis

Assume you have a simple R Markdown file called `peng.Rmd` that analyses the Palmer Penguins dataset. The file uses `pacman` for package management, `tidyverse` for data manipulation and plotting, and `knitr` for output formatting. It also references a custom LaTeX preamble file (`preamble.tex`) and a logo image (`sudoku.png`) for the PDF header.

```
---
title: "Penguins analysis"
author: "R.G. Thomas"
date: "`r format(Sys.time(), '%B %d, %Y')`"
fontsize: 11pt
geometry: "left=3cm,right=5cm,top=2cm,bottom=2cm"
output:
  pdf_document:
    keep_tex: true
    includes:
      in_header: "preamble.tex"
---

```{r include=F, echo=F}
library(pacman)
p_load(palmerpenguins, tidyverse, knitr)

opts_chunk$set(
  warning = FALSE,
  message = FALSE,
  echo = FALSE,
  results = "asis",
  dev = "pdf"
)
```

Introduction

We can work with the dataset `penguins`
included in the package `palmerpenguins`.

```{r }
library(palmerpenguins)
```
```

```
One naive approach is to split the dataset
and do three separate analyses:

```{r }
df1 <- split(penguins, penguins$species)

foo <- function(df, z) {
  df |>
    ggplot(
      aes(
        x = bill_length_mm,
        y = flipper_length_mm
      )
    ) +
    geom_point(
      aes(color = island), alpha = .5
    ) +
    geom_smooth() +
    scale_color_manual(
      values = c("purple", "green", "red")
    ) +
    theme_bw() +
    labs(
      title = paste(
        z, " Penguin Anatomy Comparison"
      ),
      x = "Flipper length",
      y = "Bill length",
      color = "Island"
    )
  plotfile_name <- paste0(z, ".pdf")
  ggsave(plotfile_name)
  cat(
    paste0(
      "\\includegraphics[height=3cm]{",
      plotfile_name, "}"
    ),
    "\n"
  )
  cat("\\vspace{1cm}", "\n")
}

bar <- df1 |> map2(names(df1), foo)
```
```

The file runs cleanly on the author's machine and produces a PDF report with three species-specific scatter plots comparing bill length and flipper length.

4.2 Sending the File to Joe

The simplest approach is to email `peng.Rmd` to Joe and ask him to render it. Joe has a Linux Mint workstation. He downloads the attachment, creates a working directory, and attempts to render.

4.3 Error 1: R Not Found

```
mkdir peng_collaboration
cd peng_collaboration
R -e "render('peng.Rmd')"
```

Linux cannot find `R`. Joe installs it:

```
sudo apt install r-base-core
```

4.4 Error 2: Function render Not Found

```
R -e "render('peng.Rmd')"
```

R loads but reports: `could not find function "render"`. Joe determines that `render` is a function in the `rmarkdown` package and installs it:

```
R -e "install.packages('rmarkdown')"
```

This itself fails due to inadequate permissions on `/usr/lib/R/library`. Joe also needs system libraries for compilation:

```
sudo apt install libssl-dev \
  libcurl4-openssl-dev unixodbc-dev \
  libxml2-dev libmariadb-dev \
  libfontconfig1-dev libharfbuzz-dev \
  libfribidi-dev libfreetype6-dev \
  libpng-dev libtiff5-dev libjpeg-dev
```

4.5 Error 3: Pandoc Version Too Old

```
R -e "library(rmarkdown); render('peng.Rmd')"
```

`pandoc version 1.12.3 or higher required`. Joe installs a newer pandoc.

4.6 Error 4: Package pacman Not Found

```
R -e "library(rmarkdown); render('peng.Rmd')"
```

`there is no package called 'pacman'`. Joe installs it:

```
R -e "install.packages('pacman')"
```

4.7 Error 5: Missing preamble.tex

```
R -e "library(rmarkdown); render('peng.Rmd')"
```

`pandoc could not find preamble.tex`. The author forgot to send the LaTeX preamble file. The file is sent, and the path is adjusted from the macOS convention to a Linux path.

Joe also needs `vim` to edit the file:

```
sudo apt install vim
```

4.8 Error 6: pdflatex Not Found

LaTeX is not installed. Joe installs `tinytex`:

```
R -e "install.packages('tinytex')"
R -e "tinytex::install_tinytex()"
```

4.9 Error 7: Missing Logo File

`pandoc error: file sudoku.png not found`. The author forgot to send the logo image referenced in the preamble. The file is sent and placed in the correct directory.

4.10 Error 8: Missing Bibliography File

`error: no bibliography file found`. Yet another file the author forgot to include. The bibliography is sent and the file path is corrected.

4.11 Error 9: Package Loading Failures

Some packages fail to load via `pacman`. Joe removes problematic packages from the script and tries again.

4.12 Finally: Success

After eight errors and several hours of debugging, Joe successfully renders the report. Every single error was caused by a mismatch between the author's environment and Joe's environment.

Figure 2: A pair of hands assembling components into a container, representing the process of packaging an analysis for reproducibility.

Packaging everything together so no piece is left behind.

5 The Docker Approach

The Docker approach eliminates every error in the preceding section by packaging the entire environment – R, packages, LaTeX, auxiliary files, and data – into a single image.

5.1 The Dockerfile

The following Dockerfile starts from the `rocker/verse` base image (which includes R, RStudio, and the tidyverse), installs additional R packages, updates the LaTeX distribution, creates a non-root user, and copies all necessary files into the image.

```
FROM rocker/verse:4
RUN apt update
RUN apt install vim -y
RUN R -e "install.packages('pacman')"
RUN R -e "install.packages('palmerpenguins')"
RUN R -e "install.packages('tidyverse')"
RUN R -e "install.packages('knitr')"
RUN R -e "install.packages('rmarkdown')"
RUN tlmgr init-usertree
RUN tlmgr update --self --all
RUN tlmgr install fancyhdr adjustbox \
  geometry titling

RUN addgroup --system joe && \
  adduser --system --ingroup joe joe
RUN chmod -R 0777 \
  '/usr/local/lib/R/site-library'
RUN chown joe:joe -R /home/joe
USER joe
WORKDIR /home/joe
RUN mkdir -p /home/joe/shr
RUN mkdir -p /home/joe/output
COPY /preamble.tex /home/joe/shr
COPY sudoku.png /home/joe/shr
COPY peng.Rmd /home/joe/shr
CMD ["/bin/bash"]
```

Key decisions in this Dockerfile:

- **`FROM rocker/verse:4`** provides R 4.x with the tidyverse and LaTeX pre-installed, reducing the number of manual installation steps.
- **Each `RUN` installs a specific package**, making the build cache effective. If you change the Rmd file, only the `COPY` layer is rebuilt.

- **`tlmgr install`** adds the exact LaTeX packages referenced by `preamble.tex`.
- **A non-root user (`joe`)** runs the analysis, which is a security best practice.
- **`COPY` commands** place the preamble, logo, and Rmd file inside the image, so nothing is missing.
- **`CMD ["/bin/bash"]`** opens an interactive shell, allowing the collaborator to inspect the environment before rendering.

5.2 Building the Image

Build the image with a tag and specify the platform for cross-architecture compatibility:

```
docker build -t rgt47/penguin_review \
  --platform=linux/amd64 .
```

5.3 Sharing the Image

There are two ways to share the image with Joe.

Option 1: Docker Hub (recommended for remote collaboration)

```
docker push rgt47/penguin_review
```

Joe pulls the image on his machine:

```
docker pull rgt47/penguin_review
```

Option 2: File transfer (useful for air-gapped networks or large images)

```
docker save rgt47/penguin_review \
  | gzip > penguin_review.tgz
```

Joe loads the image from the archive:

```
docker load -i penguin_review.tgz
```

5.4 Running the Container

Joe runs the container with a volume mount that connects a local output directory to the container's output directory. This is how rendered files are extracted from the container.

```
droot="$PWD/output"
docker run -it --rm \
  --platform linux/x86_64 \
  -v "$droot":/home/joe/output \
  rgt47/penguin_review
```

Inside the container, Joe navigates to the output directory and renders the report:

```
cd output
R -e "library(rmarkdown); \
  render('../shr/peng.Rmd')"
```

The rendered PDF appears in the local `output/` directory on Joe's machine. No errors. No missing packages. No missing files.

If Joe wants to edit the Rmd file before rendering:

```
vim ../shr/peng.Rmd
```

Vim is available because it was installed in the Dockerfile.

5.5 The LaTeX Preamble

For reference, the `preamble.tex` file that caused Error 5 in the naive approach contains the following LaTeX customisation for the PDF report header:

```
\usepackage[export]{adjustbox}
\usepackage{fancyhdr}
\usepackage{titling}

\pagestyle{fancy}

\pretitle{
\begin{flushright}
\includegraphics[width=3cm,valign=c]{
  sudoku.png}\\
\end{flushright}
\begin{flushleft} \LARGE }
\posttitle{
  \par\end{flushleft}\vskip 0.5em}
\predate{
  \begin{flushleft}\large}
\postdate{
  \par\end{flushleft}}
\preauthor{
  \begin{flushleft}\large}
\postauthor{
  \par\end{flushleft}}
\fancyfoot[L]{\currfilename}
\fancyfoot[R]{
  \includegraphics[width=.8cm]{sudoku.png}}
\fancyhead[L]{\today}
```

In the Docker image, this file is already in place at `/home/joe/shr/preamble.tex`. Joe never needs to know it exists.

5.6 Things to Watch Out For

1. **Volume mount paths must be absolute.** The `-v` flag requires an absolute path on the host side. Use `"$PWD/output"` or provide the full path explicitly.
2. **Platform flag matters for Apple Silicon.** If you build on an M1/M2/M3 Mac, the default architecture is `arm64`. Use `--platform=linux/amd64` to ensure the image runs on Intel-based Linux machines.
3. **Image size can be large.** The `rocker/verse` base image is approximately 2 GB. Adding packages increases this further. Consider `rocker/r-ver` for a smaller base if LaTeX is not needed.
4. **Non-root user permissions.** If the container writes output files as a non-root user, the host directory may need appropriate permissions. The `chmod -R 0777` in the Dockerfile is permissive; in production, use more restrictive permissions.
5. **R package versions are not pinned.** The Dockerfile installs the latest version of each package at build time. For strict reproducibility, use `renv` to pin exact package versions.

Figure 3: A quiet desk with a closed laptop and a cup of coffee, representing the conclusion of a completed workflow.

When the analysis runs on the first try, on any machine, the work is done.

5.7 Lessons Learnt

5.7.1 Conceptual Understanding

- The naive approach to sharing R code fails because it assumes the recipient's environment matches the author's environment. In practice, it never does.

- Docker eliminates environment divergence by packaging the complete computing context alongside the analysis code.
- The `rocker` project provides pre-built Docker images for R that include common dependencies, significantly reducing Dockerfile complexity.
- Volume mounts are the mechanism for extracting output from containers; without them, rendered files exist only inside the ephemeral container.

5.7.2 Technical Skills

- Building a Dockerfile for R requires installing both R packages (`install.packages()`) and system libraries (`apt install`) for packages with compiled dependencies.
- The `rocker/verse` base image includes R, the tidyverse, and a LaTeX distribution, which covers most R Markdown rendering requirements.
- `docker save` and `docker load` provide an alternative to Docker Hub for sharing images on restricted networks.
- Creating a non-root user in the Dockerfile is a security best practice that prevents the container from running processes as root.

5.7.3 Gotchas and Pitfalls

- Forgetting to include auxiliary files (preamble, images, data) in the Dockerfile `COPY` commands is the most common source of build failures.
- The `--platform` flag is essential for cross-architecture compatibility between Apple Silicon and Intel machines.
- Large Docker images take significant time to push and pull; minimising unnecessary layers and using `.dockerignore` reduces image size.
- Running `docker build` without `--no-cache` may use stale cached layers if upstream packages have been updated.

5.8 Limitations

- Docker Desktop is required on the recipient's machine, which may not be permitted in all organisational environments.
- The `rocker/verse:4` base image is approximately 2 GB, and the final image can exceed 3 GB with additional packages, making it impractical for low-bandwidth sharing.
- R package versions are not pinned in this Dockerfile; future builds may install different versions, breaking reproducibility over time.
- The Dockerfile does not use `renv` for package management; integrating `renv` would provide exact version pinning but adds complexity.
- The non-root user approach requires careful permission management when mounting volumes from the host.
- This approach does not address R code quality or correctness; Docker ensures environment consistency, not analytical validity.

5.9 Opportunities for Improvement

1. Integrate `renv` into the Dockerfile to pin exact package versions and ensure long-term reproducibility.
2. Add a `Makefile` to the image that automates the rendering step, allowing Joe to run `make render` instead of typing the full R command.
3. Use multi-stage Docker builds to separate the build environment (with compilation tools) from the runtime environment, reducing the final image size.
4. Add automated testing (e.g., `testthat`) to the Dockerfile so that the analysis is validated during the build process.
5. Create a `docker-compose.yml` file for complex analyses that require multiple services (e.g., a database alongside R).
6. Explore GitHub Actions to automatically build and push the Docker image whenever the analysis code is updated.

6 Wrapping Up

The contrast between the naive approach and the Docker approach illustrates a fundamental principle of reproducible research: the computing environment is part of the analysis. Sending an Rmd file without its environment is like publishing a paper without its methodology section.

The naive approach produced eight separate errors, each requiring Joe to diagnose, research, and fix a dependency issue. The Docker approach produced zero errors. Joe pulled the image, mounted an output directory, and rendered the report in under a minute.

What I learnt most from this experience is that reproducibility is not just about the code. The code was correct the entire time. The failures were all environmental: wrong R version, missing packages, absent files, incompatible LaTeX distributions. Docker addresses every one of these failure modes.

Main takeaways:

- The naive approach to sharing R code produced eight distinct environment errors before the analysis could run.
- Docker eliminates all eight errors by packaging the complete environment into a single image.
- The `rocker/verse` base image provides R, the tidyverse, and LaTeX, covering most R Markdown rendering requirements.
- Volume mounts allow the collaborator to extract rendered output from the container to the host machine.

7 See Also

Related posts:

- "Configure the Command Line for Data Science Development" (terminal and Docker setup)
- "Setting Up R, Vimtex, and UltiSnips in Vim" (the Vim editing environment used to author the original analysis)

Key resources:

- Running your R script in Docker (Statworx)
- Rocker Project: Docker containers for R
- Docker documentation
- renv: Project Environments for R
- Reproducible Research with R and Docker (R-bloggers)

8 Reproducibility

This workflow was developed on macOS with Docker Desktop 4.x and R 4.4. The following files constitute the complete Docker-based analysis:

File	Purpose
`peng.Rmd`	R Markdown analysis file
`Dockerfile`	Environment definition
`preamble.tex`	LaTeX header customisation
`sudoku.png`	Logo image for PDF header

To reproduce the complete workflow:

```
# Build the Docker image
docker build -t rgt47/penguin_review \
  --platform=linux/amd64 .

# Run the container with output mount
mkdir -p output
docker run -it --rm \
  --platform linux/x86_64 \
  -v "$PWD/output":/home/joe/output \
  rgt47/penguin_review

# Inside the container:
cd output
R -e "library(rmarkdown); \
  render('../shr/peng.Rmd')"
```

The rendered PDF will appear in the local `output/` directory.

9 Let's Connect

- **GitHub:** rgt47
- **Twitter/X:** @rgt47
- **LinkedIn:** Ronald Glenn Thomas

- **Email:** rgtlab.org/contact

I would enjoy hearing from you if:

- You spot an error or a better approach to any of the code in this post.
- You have suggestions for topics you would like to see covered.
- You want to discuss R programming, data science, or reproducible research.
- You have questions about anything in this tutorial.
- You just want to say hello and connect.